

Undergraduate Research Credits: Aug-Dec 2025

Using reinforcement learning for impairment correction in fibre optical communication

Medha Bhavani Girish (EE23B112)

Under the guidance of Prof. Lakshmi Narasimhan Theagarajan and Prof. Deepa Venkitesh

This report summarizes the work I have undertaken over the last 6 months under Prof. Lakshmi Narasimhan Theagarajan and Prof. Deepa Venkitesh, involving using reinforcement learning to compensate for the different impairments that occur when transmitting over an optical fiber.

I. INTRODUCTION AND MOTIVATION

Fiber optical communication systems exhibit a range of receiver impairments such as phase offset, frequency offset, crosstalk, dispersion, and other channel distortions. These impairments are often time-varying in nature, and in order to establish communication, it is necessary that these impairments are estimated and corrected for. Although adaptive equalizers to correct for each of these impairments exist, there is no single system to compensate for all of them as a single block. Correcting for all these impairments together could potentially be much more effective as well as prevent effects from one impairment cascade to the rest. Reinforcement learning offers a promising solution for this, providing an efficient framework that estimates and tracks these impairments over time. This could potentially reduce the receiver's computational complexity and increase the speed of data processing.

II. A SUMMARY OF THE DIFFERENT IMPAIRMENTS

The below section summarizes the cause of different impairments in optical fibers and the existing methods for correction, along with how these impairments are modeled.

A. Chromatic Dispersion

Chromatic dispersion occurs when different spectral components of a guided mode propagate with distinct group velocities. Chromatic dispersion leads to the pulses broadening during transmission, spreading their energy over adjacent symbol periods and creating intersymbol interference (ISI). The model chromatic dispersion, its frequency response is computed as

$$G(\omega) = \exp\left(j \frac{D\lambda_c^2}{4\pi c} L \omega^2\right),$$

where D is the chromatic dispersion parameter of the fiber, λ is the wavelength of the optical carrier, c is the speed of light in vacuum, L is the length of the fiber, and ω is the angular frequency based on the number of samples per symbol and the symbol rate.

The CD compensation module is implemented at the receiver's end using a static compensation filter, since CD is well-behaved in time and equally affects both polarization orientations.

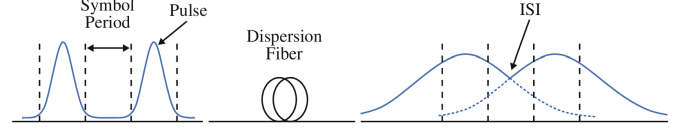


Fig. 1. Pulse broadening leading to ISI in Chromatic Dispersion

B. Polarization Effects

Single mode fibers support the propagation of 2 orthogonally polarized modes, which ideally have the same propagation constant and are degenerate. However, the non perfect circularity of the core and cladding cause a loss of degeneracy which leads to pulse distortions at the receiver's end. The fiber acquires birefringence, which is a difference between the refractive indices of the 2 polarization modes. This causes a temporal spread of pulses, called polarization mode dispersion (PMD). Pulses in orthogonal polarizations acquire distinct group velocities, and a mutual delay called the differential group delay (DGD) develops between projections of the linearly polarized signal onto its orthogonal states, given by

$$\tau = \frac{l\Delta\beta}{\omega_0},$$

where l is the length, $\Delta\beta$ is the difference in propagation constants, and ω_0 is the angular carrier frequency.

This impairment is modeled by representing the electric field as a 2-d complex vector, and showing the transformation due to polarization by a frequency dependent matrix.

$$\begin{bmatrix} E_{o,x}(\omega) \\ E_{o,y}(\omega) \end{bmatrix} = H(\omega) \begin{bmatrix} E_{i,x}(\omega) \\ E_{i,y}(\omega) \end{bmatrix}.$$

A single birefringent section is modeled as

$$H^{(i)}(\omega) = V^{(i)} \Lambda^{(i)}(\omega) U^{(i)H},$$

where $U^{(i)}$ and $V^{(i)}$ are 2×2 unitary matrices representing random polarization coupling, and

$$\Lambda^{(i)}(\omega) = \begin{bmatrix} e^{j\omega\tau^{(i)}/2} & 0 \\ 0 & e^{-j\omega\tau^{(i)}/2} \end{bmatrix}$$

incorporates the DGD $\tau^{(i)}$ of that segment. The coupling matrices are generated by applying singular value decomposition (SVD) to random complex Gaussian matrices.

Long fibers are modeled as a cascade of N such birefringent sections, each with different random polarization coupling unitary matrices. The overall transfer function of the fiber is then the product of the individual matrices across sections.

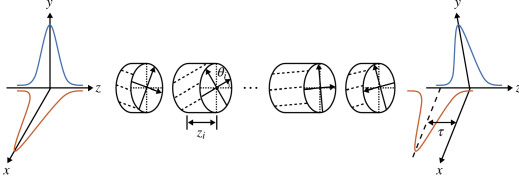


Fig. 2. Effect of polarization mixing across sections of fiber

Because polarization effects fluctuate stochastically in real-world deployments due to different environmental changes, coherent receivers dynamically correct for them, employing adaptive polarization-tracking equalizers to mitigate PMD-induced distortions.

C. Phase Noise

Phase noise arises from random temporal fluctuations in the phase of the optical carrier. These fluctuations are typically modeled as $\phi[k] = \phi[k-1] + \Delta\phi[k]$, with $\Delta\phi[k]$ being a zero-mean Gaussian variable whose variance is proportional to the combined laser linewidth. Phase noise causes constellation rotation and degradation in coherent detection. Phase noise can be corrected using the LMS algorithm below to track and compensate for phase drift.

D. Frequency Offset

Frequency offset occurs when the transmitter and receiver oscillators have a difference in frequencies, that causes a phase rotation of the received signal. The impairment is modeled as the multiplication of the received samples by $e^{j2\pi\Delta f n T_s}$, where Δf is the carrier frequency mismatch and T_s is the sampling interval. A method for rectifying this in the case of QPSK involves raising the received signal to the fourth power such that the modulation is removed and the resulting phase contains only the term $4(2\pi\Delta f n T_s)$. Taking the unwrapped phase and dividing by four provides an estimate of the frequency offset, which can be removed.

III. CURRENT ALGORITHMS FOR CORRECTION

LMS (Least Mean Squares) Algorithm: The LMS algorithm is an adaptive filtering method that minimizes the mean squared error between the received signal and the transmitted signal, which is then used to update the filter coefficients via stochastic gradient descent.

Constant Modulus Algorithm (CMA): The Constant Modulus Algorithm relies on the constant modulus property of certain modulations such as QPSK, where the transmitted signal has a fixed amplitude. The key idea is that ISI introduces variations in the amplitude of the received signal, and by enforcing a constant amplitude at the equalizer output, CMA reduces ISI.

CMA defines a cost function based on the deviation of the output signal's modulus from a reference dispersion constant R_d , which is then used for stochastic gradient descent:

$$J_{\text{CMA}} = \mathbb{E} [(R_d - |y[k]|^2)^2]$$

The equivalent error signal used in stochastic gradient descent is:

$$e_{\text{CMA}}[k] = (R_d - |y[k]|^2) y[k]$$

The filter coefficients are updated as:

$$w[n+1] = w[n] + \mu x[n] e_{\text{CMA}}^*[k]$$

CMA works best with constant modulus signals like in QPSK, but it can handle QAM reasonably well as well. There are also versions of the algorithm which involve mapping it onto one of 3 radii rather than a single fixed one, which takes non-constant amplitudes into account. One point to note here is that CMA corrects only the modulus- the phase offset will need to be corrected separately.

IV. MODELING USING REINFORCEMENT LEARNING

A. Multi-Armed Bandits

The Multi-Armed Bandit is a model in reinforcement learning for situations wherein a decision-maker must choose between several options with uncertain rewards. The goal is to maximize the cumulative reward over time by deciding which arm to play at each step. Consider a set of K arms, each associated with an unknown reward distribution. At each time step t , the agent selects an arm $a_t \in \{1, 2, \dots, K\}$ and receives a reward r_t drawn from the corresponding distribution. The objective is to maximize the expected cumulative reward:

$$R_T = \sum_{t=1}^T r_t$$

where T is the total number of rounds. The fundamental challenge in MAB is finding a balance between exploration and exploitation to maximize long-term gains.

There are different policies that can be used to select the next arm. The following are the ones we experimented with:

- 1) **Epsilon-Greedy:** The agent explores a random arm with probability ϵ , and exploits the current best arm with probability $1 - \epsilon$.
- 2) **Upper Confidence Bound (UCB):** Selects the arm with the highest upper confidence bound on the estimated reward:

$$a_t = \arg \max_i \left(\hat{\mu}_i + c \sqrt{\frac{\ln t}{n_i}} \right)$$

where a_t is the arm chosen at time step t , $\hat{\mu}_i$ is the estimated mean reward of arm i , n_i is the number of times arm i has been selected so far, t is the total number of rounds elapsed, and c is a tunable parameter controlling the degree of exploration.

- 3) **Thompson Sampling:** Uses a Bayesian posterior to sample the expected reward for every arm and chooses the arm with the highest sampled value. However, this policy requires rewards to follow a fixed family of distributions for every arm, so it did not seem suitable in our problem statement.

Formulating the problem using MABs: To formulate our problem statement as a multi-armed bandit, we set different sets of coefficients for the adaptive equalizer as the arms of

our bandit, and the reward as the negative of the same error we use in the CMA algorithm. Focusing on correcting just polarization effects from QPSK symbols, we require 4 adaptive filters- one for each channel in a 2x2 MIMO setup. If each of these filters have 11 filter taps, which is what the previously implemented CMA algorithm uses, then even by discretising the range of values to select filter tap coefficients to just 5 values, we are left with $5^{4 \cdot 11}$ choices of arms, which is not feasible to implement.

Before shifting to alternate implementations, we first look to implement a proof of concept for the UCB algorithm considering a much smaller space for filter tap coefficients. For this, we consider just 3 filter tap coefficients, and 3 fixed values that each of these coefficients can take. These values are based on the optimal coefficients found from the CMA algorithm, taking one configuration of coefficients to be the optimal one, and the other configurations randomly. This is used as a bench mark for convergence.

Therefore, we initialise 4 filters, each having 3 taps, and generate all possible filter coefficient combinations for these, which is 3^{12} in this case. This serves as the space in which we search for which arm, or which filter to select next. We have a dataset of 100 symbols that are impaired with polarization effects, that belong to the QPSK encoding. We also define basis points, which are the ideal constellations. Whenever an arm is chosen, that is, coefficients are selected for each of the 4 channels, the X and Y symbols from the dataset are convolved with the filters to give the rectified symbols. Based on this, the error is computed in the same way as CMA, i.e. by mapping the symbols to a fixed amplitude. We begin by playing every arm once to get an estimate of its reward, and then the UCB algorithm is used to select the arm having the highest score- initially preferring exploration, but then focusing on exploitation. We find that after about a 1000 rounds, the UCB algorithm converges to the best set of coefficients, and corrects for the polarization effect. It is to be noted that since the CMA error accounts only for offsets in modulus and not phase, even the correct symbols will have some rotation from the ideal symbols.

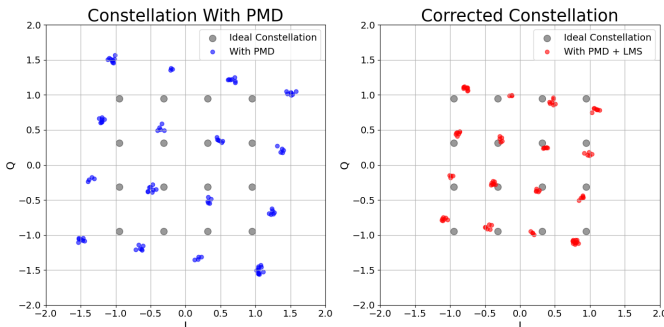


Fig. 3. Received polarized constellations before and after correction

While this toy exercise with just 3 filter taps and 5 options for the coefficient values served as a proof that UCB converges, it is not feasible when we have a much higher number of arms- as would be the case in a practical implementation. For this, we shift our attention to Q-Learning, in which we can define

our actions as an increment or decrement being made to the previously set coefficients- allowing us to try a much wider range of values, as opposed to MAB where the coefficients we chose were independent of the previous ones and belonged to a fixed set of discrete values.

B. Q-Learning

Q-learning extends the MAB model into sequential decision-making problems where actions influence future states and rewards. In contrast to MAB where every decision is independent of the past, Q-learning considers both immediate and future rewards by maintaining a 'Q-value' for every state-action pair, which estimates the expected cumulative reward of taking a particular action when in a particular state and then following an optimal policy after that.

Q-values are updated iteratively using the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

where s is the current state, a is the chosen action, r is the reward received after taking action a , s' is the next state, α is the learning rate, and γ = discount factor for future rewards. Here, a Q-table is used as a lookup table that stores Q-values for every state-action pair. At each time step, when a particular action is chosen for a given state, the reward is sampled, and the q-value is accordingly updated in the table. After many episodes, the q-values converge to the expected cumulative reward for that state-action pair, and then the agent can accordingly choose the best action in any state.

We additionally use a replay buffer, which is a memory that store's the agent's past experiences of sampled rewards and next states for a given state-action pair. By sampling multiple past experiences of the same (s,a) pair instead of simply relying on the most recent one, the algorithm gets a more reliable estimate of the cumulative reward.

In our case, where we have an exponential number of possible state-action pairs, storing q-values for all of these in a table is not practical, and would consume a lot of memory. Instead, we can implement a Deep Q-Learning Network (DQN) which trains a neural network to predict the reward for a given state-action pair, instead of explicitly storing these values in a table.

Formulating the problem using Q-Learning:

- The state here we define is a vector of 24 elements- 4 channels (pxx, pyy, pxy, pyx), 3 filter taps per channel, and a real and imaginary part for each tap.
- The action is a number between 1 and 72- there are 3 actions corresponding to each element in the 24-length vector- incrementing, decrementing the element, or keeping it constant.
- The reward is a scalar we predict based on how close the symbols are after convolving it with the filter coefficients given by that particular action to the ideal constellations, and the next state represents the updated coefficients after the chosen action.
- The policy used is the greedy-epsilon method, but with a decay- so initially the probability of exploration is high, but with time as we gain more knowledge about our

system, we start exploiting and always selecting the best action based on our knowledge.

- The Q-network is trained by taking the MSE between the predicted Q-values from the Q-network and the target Q-values which is calculated as the sum of the sampled reward from this state and a discounted value of the next state's Q-value.
- We additionally first train this DQN using a teacher training mechanism- During the exploitation rounds, instead of using the action recommended by the Q-network, we instead compute the action that the classical CMA algorithm would choose, i.e., the update in coefficients that gives the largest gradient magnitude, and use this to get to the next state. We initially train our agent using this, helping it collect more information early in the training process. After a fixed number of episodes, we let the agent behave autonomously based on its own learned policy.

Parameters set:

- $\epsilon = 1.0$, $\epsilon_{\text{new}} = \max(\epsilon \times 0.995, 0.05)$
- Q-network:
 - Input dimension: 24 (state vector)
 - Hidden layers: 3 fully connected layers, each of size 64, LeakyReLU after every layer
 - Output dimension: 72 (one Q-value per discrete action)
- Learning rate = $5e-4$
- Replay buffer capacity = 10000
- Soft target update: $\tau = 0.01$

$$(\theta_{\text{target}} \leftarrow \tau \theta_{\text{online}} + (1 - \tau) \theta_{\text{target}})$$

- Number of episodes = 400, 1000 timesteps/episode.
- Filter update step size delta = $1e-3$

Findings: While the classical CMA algorithm converges efficiently, the DQN loss does not converge with the above parameters, and as a result, the filter taps aren't optimized by the end of 400 episodes. Some hyperparameter tuning needs to be carried out, and this is a work in progress- the model is still being developed.

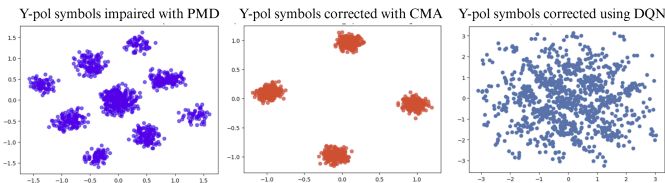


Fig. 4. Results of correcting 4-QPSK symbols impaired with PMD using CMA and DQN- the Q-learning method does not give good corrections

V. FUTURE STEPS AND ALTERNATE IMPLEMENTATIONS

In addition to Q-Learning, a potential formulation of this problem could be with combinatorial multi-armed bandits. The number of actions for a combinatorial MAB are exponentially large- each action being a vector of several features. This fits into our framework, where an action may be updating some number of filter tap coefficients to correct for PMD, along

with other features to correct for other forms of impairments. Here, rather than selecting a single arm, the learner chooses a subset of arms- and the learner's goal is to learn over time which arm subsets give the highest expected reward. The Algorithmic Configuration framework, proposed in the paper below, repeatedly calls a combinatorial successive elimination subroutine that evaluates subsets of configurations, or actions, in parallel, to rank them and aggressively discard the poorly performing ones in the beginning, while retaining the good ones. By scheduling epochs with decreasing subset sizes and less aggressive elimination, the learner gradually converges on the best action with high probability. This could further be modeled as a correlated combinatorial multi-armed bandit problem- since the actions here aren't independent of each other. A set of filter coefficients that are quite close to each other would give similar performances, and this fact can be exploited to learn about a set of actions, or filter coefficient configurations, by playing just one. This would also ideally be a restless MAB- since the equalizer is adaptive, it means the 'best arm', or the optimal set of coefficients change with time. The next course of action after implementing the Q-Learning network involves looking into this implementation of combinatorial correlated restless multi-armed bandits.

This concludes the work I've undertaken as part of UGRC from July to December 2026. I am deeply grateful to Prof. LNT and Prof. Deepa, and Mandeep as well, for their guidance- I learnt a lot.

References

- 1) J. Brandt, E. Schede, V. Bengs, B. Haddenhorst, E. Hüllermeier, and K. Tierney, *AC-Band: A Combinatorial Bandit-Based Approach to Algorithm Configuration*.
- 2) S. Wang, L. Huang, and J. C. S. Lui, *Restless-UCB: An Efficient and Low-complexity Algorithm for Online Restless Bandits*.
- 3) S. Gupta, S. Chaudhari, G. Joshi, and O. Yağan, *Multi-Armed Bandits with Correlated Arms*.
- 4) W. Chen, Y. Wang, and Y. Yuan, *Combinatorial Multi-Armed Bandit: General Framework, Results and Applications*.
- 5) N. Silva, H. Werneck, T. Silva, A. C. M. Pereira, and L. Rocha, *Multi-Armed Bandits in Recommendation Systems: A Survey of the State-of-the-Art and Future Directions*.